

CS 527 Research Project Report: Group-15

Detecting Logic Drift in Paper Trading Agentic Systems

Jung In Chang, Hsien-Hao Li
University of Illinois at Urbana-Champaign
Chicago, Illinois
jichang2@illinois.edu
hl128@illinois.edu

Abstract

The integration of Agentic AI into financial decision-making introduces unique challenges regarding behavioral stability and "logic drift", the unintended divergence of an agent's decision-making logic under volatile or adversarial inputs. Current validation techniques often rely on backtesting for profitability, which fails to capture the structural stability of the agent's logic. This project proposes a framework for analyzing the robustness of deterministic, agentic paper-trading systems by applying novel mutation operators to sentiment inputs and utilizing regression testing on decision trajectories. Our approach bridges this gap by focusing on the software engineering aspect of agentic reliability, aiming to quantify logic drift and identify susceptibility to manipulation against predefined risk oracles.

1 Introduction

The rapid proliferation of Agentic AI in high-stakes financial environments has transformed automated trading from static execution to dynamic, multi-agent decision-making. These systems frequently ingest unstructured, high-velocity data, such as social media sentiment and real-time market signals, to inform autonomous trading actions. Ensuring the reliability and safety of these agents is paramount, as a single failure in decision logic can lead to significant financial loss or systemic market instability.

State-of-the-art techniques for validating trading agents primarily rely on financial backtesting, where performance is measured by profitability and Sharpe ratios over historical datasets. In the broader field of agentic programming, researchers have begun exploring trajectory validation and formal verification to ensure agents adhere to predefined constraints. Recent efforts in LLM-based agent evaluation focus on "correctness" based on task completion and benchmarking against static datasets to ensure the agent follows human-provided instructions.

Despite these advancements, a significant research gap exists regarding "logic drift", the unintended divergence of an agent's internal decision-making logic under volatile or adversarial inputs. While the machine learning community has extensively studied "concept drift" regarding model accuracy [2], our focus on "logic drift" specifically addresses the structural stability of the agentic decision trajectory. Existing financial backtesting fails to distinguish between a "lucky" profit and a "robust" decision process, masking structural instabilities. Furthermore, current software engineering (SE) techniques for agents often treat them as black boxes, failing to quantify how specific input perturbations, such as "pump-and-dump" sentiment spikes, cause an agent to deviate from its intended risk management profile.

We propose a framework to bridge this gap by treating the agentic trading system as a stateful, deterministic software entity subject to rigorous mutation and regression testing. Our approach employs metamorphic relations—where a specific change in sentiment input should result in a predictable shift in trading action—to validate behavior without a perfect ground-truth oracle [1]. By applying novel mutation operators to sentiment artifacts, we can detect logic drift by computing divergence metrics between baseline and perturbed decision trajectories.

Our experimental plan involves developing a multi-agent paper-trading system that executes trades based on replayed, time-indexed sentiment and market artifacts. We will evaluate the system's robustness by injecting controlled perturbations—such as sentiment intensity scaling and signal delays—and measuring the "Mutation Detection Rate" of our testing suite. This builds on the principles of mutation analysis traditionally used to evaluate test suite thoroughness [3], now applied to the domain of agentic environmental signals. We will specifically analyze the agent's susceptibility to adversarial manipulation and its ability to adhere to hard risk oracles under distribution shifts.

Through these experiments, we expect to demonstrate that our framework can successfully identify logic drift before it manifests as catastrophic financial failure. We anticipate that our mutation testing approach will reveal specific fragility regimes where the agent's decision-making becomes hyper-sensitive to external noise, providing a more granular safety metric than traditional profitability-based evaluation.

The main contributions of this paper are as follows:

- **Novel Definition:** We formally define and characterize "logic drift" in the context of deterministic agentic trading systems.
- **Testing Framework:** We develop a three-layered architecture for reproducible trajectory logging and automated regression testing.
- **Mutation Operators:** We introduce a suite of sentiment-specific mutation operators designed to simulate adversarial market environments.
- **Divergence Metrics:** We propose the Action Difference Ratio and Exposure Drift Score as quantitative metrics for measuring behavioral stability.

2 Proposed Technique

2.1 Approach Overview

The proposed technique utilizes a three-layered architecture designed to decouple environmental inputs from decision logic to ensure deterministic evaluation. The system ingests social media

discourse to generate signal artifacts, executes a non-stochastic trading policy, and evaluates behavioral stability through trajectory divergence analysis. This structure allows us to treat the agentic trading system as a stateful software entity that can be rigorously validated using mutation and regression testing techniques.

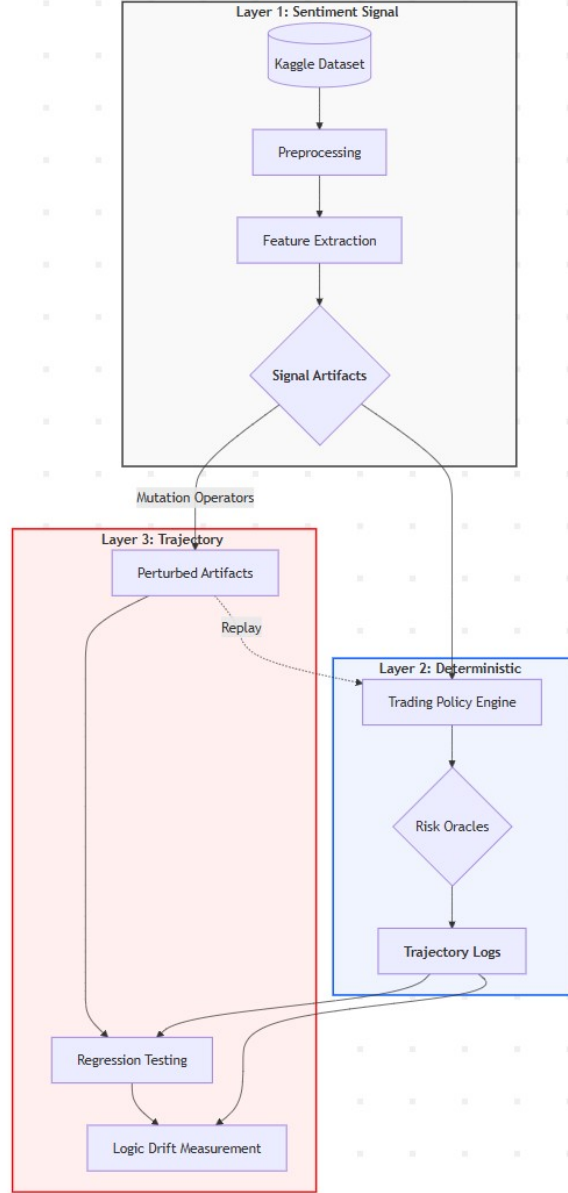


Figure 1: Overview of the proposed three-layered architecture for detecting logic drift, showing the flow from Kaggle data ingestion to trajectory analysis.

2.1.1 Layer 1: Sentiment Signal Generation. This layer transforms unstructured data into reproducible, time-indexed artifacts. We leverage the *Cryptocurrency Tweets Dataset* from Kaggle to provide a high-volume historical record of social media discourse.

- **Preprocessing:** Tweets are cleaned, categorized by cryptocurrency (e.g., BTC, ETH), and mapped to discrete time windows.
- **Feature Extraction:** For each window, we compute a structured feature vector including polarity scores, intensity metrics, and volume indicators.
- **Deterministic Artifacts:** These features are aligned with market snapshots to create replayable artifacts, ensuring that every baseline test run is identical.

2.1.2 Layer 2: Deterministic Decision Layer. The decision layer executes the core trading logic. To support software-level regression testing, this layer eliminates stochasticity; given identical inputs, the policy will always yield the same trajectory.

- **Trading Policy Engine:** A rule-based engine that evaluates sentiment signals against predefined risk management profiles.
- **Risk Oracles:** The system enforces hard constraints, such as position caps and drawdown limits, which act as safety oracles during execution.
- **Trajectory Logging:** The layer logs the full decision trajectory, including inputs, internal agent states, and actions (buy, sell, or hold).

2.1.3 Layer 3: Trajectory Analysis Layer. This layer serves as the testing suite for detecting logic drift and assessing robustness.

- **Mutation Testing:** We apply mutation operators to the Layer 1 artifacts—such as scaling sentiment intensity or injecting signal delays—to simulate adversarial market environments.
- **Regression Testing:** The system replays the original signal artifacts to establish a baseline and then replays mutated artifacts to capture perturbed trajectories.
- **Logic Drift Measurement:** By comparing these trajectories, we compute divergence metrics like the Action Difference Ratio to identify structural shifts in decision logic caused by external perturbations.

3 Experimental Evaluation

3.1 Research Questions

Through this research project, we aim to answer the following research questions (RQs) regarding the stability of agentic trading systems:

- **RQ1: Sensitivity to Sentiment Volatility.** To what extent do small perturbations in sentiment intensity (e.g., 20% scaling) lead to non-linear divergence in the agent’s trading trajectory?
- **RQ2: Robustness Against Signal Latency.** How does the injection of temporal jitter (signal delay) impact the agent’s ability to adhere to its deterministic risk oracles?
- **RQ3: Detection of Adversarial Logic Drift.** Can our mutation testing framework successfully identify behavioral

shifts caused by synthetic "pump-and-dump" signals that traditional backtesting would otherwise ignore?

3.2 Experimental Setup

Our evaluation utilizes the *Cryptocurrency Tweets Dataset* from Kaggle, which provides 1.6 million tweets aligned with historical Bitcoin and Ethereum market movements. We will isolate specific high-volatility time windows to serve as our baseline input artifacts. The system is implemented in Python, utilizing a multi-agent framework to ensure that Layer 1 signal artifacts and Layer 2 decision trajectories are fully recorded and replayable.

3.3 Evaluation Plan

To answer the research questions, we will execute a three-stage evaluation plan:

3.3.1 Phase 1: Baseline Trajectory Generation. We will execute the agent on a fixed set of historical data to establish a baseline trajectory T_{base} . This trajectory includes all actions (Buy, Sell, Hold) and internal state transitions logged at each time step.

3.3.2 Phase 2: Mutation and Replay. We will apply a suite of mutation operators to the baseline sentiment artifacts:

- **Intensity Mutation:** Scaling sentiment scores by factors of 0.5 and 2.0.
- **Temporal Mutation:** Shifting signal arrival times by 5, 10, and 30 minutes.
- **Adversarial Injection:** Inserting high-sentiment "noise" into neutral market regimes.

The agent will be re-run (replayed) against these mutated artifacts to generate perturbed trajectories T_{mutant} .

3.3.3 Phase 3: Divergence Analysis. We will quantify logic drift using the following metrics:

- **Action Difference Ratio (ADR):** The percentage of time steps where $A_t^{mutant} \neq A_t^{base}$.
- **Oracle Violation Rate:** The frequency at which mutated inputs cause the agent to attempt actions that violate hard risk constraints (e.g., position caps).
- **Mutation Detection Rate:** The percentage of mutations that result in an ADR exceeding a defined stability threshold τ .

4 Related Work

4.1 Agentic AI and Trajectory Validation

The verification of autonomous agents often focuses on goal-reachability and safety constraints. Recent literature has shifted from evaluating static model outputs to analyzing the "trajectory" of agentic workflows—the sequence of internal states and external actions taken to achieve a task. Our project extends this by focusing on financial agents, where trajectory stability is a prerequisite for safety under volatile market conditions.

4.2 Mutation and Metamorphic Testing

Mutation testing is a well-established software engineering technique used to evaluate the robustness of test suites by introducing

deliberate faults into source code [3]. Our research adapts this methodology to the input domain; by mutating environmental sentiment signals rather than code, we evaluate the robustness of the decision logic itself. This approach is closely related to Metamorphic Testing (MT), which addresses the "oracle problem" by defining relations between input changes and expected output shifts [1]. We utilize MT principles to validate that trading agents react consistently to scaled sentiment intensities without needing a perfect ground-truth trading oracle.

4.3 Concept Drift vs. Logic Drift

While the machine learning community has extensively researched "concept drift"—where the statistical properties of a target variable change over time—the focus is typically on model accuracy and retraining [2]. Our work introduces the concept of "logic drift," which specifically targets the structural stability of the agent's software-driven decision logic. This allows us to quantify how adversarial inputs like "pump-and-dump" spikes cause an agent to violate its original risk-management profile, even if the underlying model remains accurate in a narrow sense.

4.4 Reliability in Financial Systems

Standard financial validation relies heavily on backtesting to measure alpha and profitability. However, these methods often ignore the software engineering aspects of reliability and deterministic execution. By integrating software-level regression testing with paper-trading, our project bridges the gap between financial modeling and agentic software safety.

References

- [1] Tsong Yueh Chen, Shing Chi Cheung, and S. M. Yiu. 1998. Metamorphic testing: a new approach for generating next test cases. *Technical Report, HKUST-CS98-01* (1998).
- [2] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A survey on concept drift adaptation. *ACM computing surveys (CSUR)* 46, 4 (2014), 1–37.
- [3] Milos Ojdanic, Ezekiel Soremekun, Renzo Degiovanni, Mike Papadakis, and Yves Le Traon. 2023. Mutation Testing in Evolving Systems: Studying the Relevance of Mutants to Code Evolution. *ACM Transactions on Software Engineering and Methodology* 32, 1 (2023), 1–39.